

Charotar University of Science and Technology (CHARUSAT)

Faculty of Technology and Engineering (FTE)

Unit-wise CO Mapping with Justification

Subject	Object Oriented Programming (CEUE203)
Semester	3rd
Teaching Scheme	Theory 2 hrs/week · Practical 2 hrs/week · 4 Credits
Academic Year	2026–27

1. Course Outcomes (Reference)

CO1: Java fundamentals + OOP principles (inheritance, polymorphism, abstraction, encapsulation).

CO2: Inheritance, interfaces, and packages for modular, maintainable, reusable code.

CO3: Exception handling for fault-tolerant, stable applications.

CO4: Multi-threaded applications for optimized performance and resource utilization.

CO5: File handling, NIO.2, and the Stream API for efficient I/O and data-stream processing.

CO6: Collection Framework and Generics for robust, type-safe data management.

2. Unit-wise CO Mapping

Each syllabus unit is mapped to the Course Outcome(s) it primarily develops. Contact hours and evaluation weightage are shown as per the syllabus.

Unit	Unit / Topic	Hrs	Wt (%)	Mapped CO
1	Fundamental of Programming in Java	02	06	CO1
2	Class Fundamentals	04	14	CO1
3	String Handling	02	06	CO1
4	Inheritance, Interfaces & Packages	05	17	CO2 (reinforces CO1)
5	Exceptions Handling	02	06	CO3
6	Multithreaded Programming	05	17	CO4
7	File NIO	04	14	CO5
8	Collection Framework and Generics	06	20	CO6
	Total	30	100	

3. Justification of Unit–CO Mapping

Unit	Mapped CO	Justification
Unit 1	CO1	This unit introduces Java’s execution model (the bytecode and JVM), the language’s defining characteristics (the Java Buzzwords), and recently added features such as switch expressions and records. It establishes the conceptual and historical foundation that CO1 targets — a comprehensive understanding of Java fundamentals on which all later programming is built.
Unit 2	CO1	Defining classes and objects, access modifiers, constructors and overloading, the this/this() mechanism, the Object class methods, memory management (stack vs. heap, garbage collection), and annotations/reflection collectively build the student’s grasp of encapsulation and the core object-oriented principles named in CO1. It is the practical realization of the OOP fundamentals CO1 requires.
Unit 3	CO1	String and StringBuilder/StringBuffer usage, string-manipulation techniques, regular expressions, and tokenization are essential data-handling fundamentals of the Java language. Mastery of these consolidates the strong programming foundation defined in CO1 and prepares students for text processing in later units.
Unit 4	CO2 (reinforces CO1)	The super keyword, dynamic method dispatch, abstract classes, interfaces (including functional and marker interfaces and the Interface Segregation Principle), packages, JAR files, and the Java Module System are exactly the constructs CO2 requires to create modular, maintainable, and reusable code. The runtime polymorphism and abstraction covered here also reinforce the OOP principles stated in CO1.
Unit 5	CO3	Checked and unchecked exceptions, try-catch-finally, throw/throws, try-with-resources, custom exceptions, and exception propagation are precisely the mechanisms CO3 targets. The unit directly develops the ability to construct fault-tolerant applications that manage errors and maintain program stability.
Unit 6	CO4	The thread lifecycle, the Thread class and Runnable interface, synchronization and inter-thread communication, thread priority and scheduling, deadlock and thread-safety handling, and the Executor/ExecutorService and Fork/Join frameworks map directly to CO4’s goal of developing multi-threaded applications that optimize performance and resource utilization.
Unit 7	CO5	The Path interface and Files class, file attributes, byte and character streams, serialization, and the NIO fundamentals (Buffer, Channel, Selector, directory streams, and file walking) correspond directly to CO5’s file-handling and NIO.2 capabilities, enabling efficient file operations and data-stream processing in data-driven applications.
Unit 8	CO6	The Collection interface with the List, Set, and Map hierarchies, iteration/sorting/searching, thread-safe collections, and generics (generic classes and methods, bounded type parameters, wildcards, and type erasure) map directly to CO6. The unit develops the ability to build robust, efficient programs that manage data with type safety and reduced runtime errors.

4. CO Coverage Summary

Every Course Outcome is addressed by at least one unit, and every unit maps to a Course Outcome — confirming complete, traceable coverage with no orphaned content.

CO	Addressed in
C01	Units 1, 2, 3
C02	Unit 4
C03	Unit 5
C04	Unit 6
C05	Unit 7
C06	Unit 8